

实验 8 文件系统调用

8.1 实验目的

- (1) 理解文件存储空间的管理、文件的物理结构和目录结构；
- (2) 掌握与文件有关的系统调用。

8.2 预备知识

文件是数据的集合，系统文件不仅包含文件中的数据，而且还有文件系统的结构。文件系统负责在外存上管理文件，并把对文件的存取、共享和保护等手段提供给操作系统和用户。文件系统不仅方便了用户使用，保证了文件的安全性，还可以大大提高系统资源的利用率。

Linux 最早的文件系统是 Minix，它受限甚大且性能低下。其文件名最长不能超过 14 个字符（虽然比 8.3 文件名要好）且最大文件大小为 64M 字节。64M 字节看上去很大，但实际上一个中等的数据库将超过这个尺寸。第一个专门为 Linux 设计的文件系统被称为扩展文件系统（Extended File System）或 EXT。它出现于 1992 年四月，虽然能够解决一些问题但性能依旧不好。1993 年扩展文件系统第二版或 EXT2 被设计出来并添加到 Linux 中。它是本章将详细讨论的文件系统。

将 EXT 文件系统添加入 Linux 产生了重大影响。每个实际文件系统从操作系统和系统服务中分离出来，它们之间通过一个接口层：虚拟文件系统或 VFS 来通讯。

VFS 使得 Linux 可以支持多个不同的文件系统，每个表示一个 VFS 的通用接口。由于软件将 Linux 文件系统的所有细节进行了转换，所以 Linux 核心的其它部分及系统中运行的程序将看到统一的文件系统。Linux 的虚拟文件系统允许用户同时能透明地安装许多不同的文件系统。

对用户而言，应用程序可以通过文件系统提供的系统调用来使用文件资源，在用户程序中有关文件的操作主要有：创建文件、打开文件、读写文件、关闭文件等。

8.3 实验内容

1. 创建、打开、关闭和读写文件；
2. 模拟实现文件的复制。

8.4 实验指导

1.创建文件

系统调用 creat 创建一个新文件或重写一个旧文件，并将文件描述符 fd 返回给用户程序，用户利用文件描述符对文件进行读写。

创建文件系统调用格式为：

```
int creat(path, smode);
```

```
char* path;
int smode;
```

该系统调用的返回值是文件描述符。其中 `path` 为创建的文件名（包含全路径）；`smode` 为文件的实际权限，在文件修改模式命令 `chmod` 中，用户对文件读、写、执行的访问权 `mode` 用 3 位 8 进制表示，例如 `mode=751` 说明文件的所有者有读、写、执行权，同组用户有读、执行权，其他用户只有执行权。

系统按照 `smode` 指定的权限创建文件，如果创建成功，则返回文件描述符，否则返回-1。如果文件已经存在，重写一个旧文件意味着要将文件中原有内容清掉。

说明：

3 位 8 进制表示法：`r`、`w`、`x` 表示读、写、执行权。

每次使用时所有者、组、其他用户的访问权分别用相应的数字带入。记住这些数字的基本规律：`001 (x)`、`010 (w)`、`100 (r)` 是基本数字；3 由 `1+2` 构成 (`x+w`)，5 由 `1+4` 构成 (`x+r`)，6 由 `2+4` 构成 (`w+r`)，7 由 `1+2+4` 构成 (`x+w+r`)。

2.打开文件

文件创建以后，必须用系统调用 `open` 将文件打开之后才能对文件进行读写操作。

系统调用的格式为：

```
int open(path, rwmode);
char* path;
int rwmode;
```

该系统调用返回值为打开文件的描述符。如果文件打开成功，则返回文件描述符，否则返回-1。其中 `path` 为含有全路径的要打开的文件名；`rwmode` 为头文件 `fcntl.h` 中定义的对打开文件的访问模式：

```
rwmode = 0 表示可读，    O_RDONLY;
rwmode = 1 表示可写，    O_WRONLY;
rwmode = 2 表示可读可写，O_RDWR;
```

3.关闭文件

当用户在文件使用完之后不再需要时，用系统调用 `close` 断开用户程序与文件之间的通路，关闭该文件。

系统调用格式为：

```
int close(fd);
int fd;
```

其中 `fd` 为文件描述符，根据 `fd` 从用户文件描述符表中得到指向文件表项的指针 `fp`，由 `fp` 得到文件表项中的 `f.count`，并对 `f.count` 作减 1 操作。操作结果如果不为 0，表示还有进程在使用该文件，此时不能回收文件表项；操作结果为 0，表示无进程使用该文件，此时将此文件表项置为空。

4.读文件

当文件打开后，可以用系统调用 `read` 对文件进行读操作。

系统调用格式为：

```
int read(fd, buffer, count);
int fd;
```

```
char* buffer;
unsigned count;
```

其中, fd 是 open 返回的文件描述符; buffer 是一个缓冲区, 用于存放所读的数据; count 是要读的字节数。如果系统调用成功, 则返回实际读取的字节数; 如果读的时候, 读指针已经到达文件末尾, 但还没有读够指定的字节数 count, 也立即停止, 所以返回值可以小于等于 count。在 read 系统调用中如果没有指定要读的字节数, 则通常是读 1 个字节。

5. 写文件

写文件系统调用 write 是在文件被 open 系统调用打开后, 从缓冲区 buffer 将指定字节的内容写到由 open 返回的文件描述符所指定的文件中。

系统调用格式为:

```
int write(fd, buffer, count);
int fd;
char* buffer;
unsigned count;
```

其中, fd 是 open 返回的文件描述符, 该文件存放由 write 写入的内容; buffer 是一个缓冲区, 用于存放要写的内容; count 是要写的字节数。

8.5 参考源代码

注意文件系统调用中的路径参数, 根据实验环境具体填写 (可以使用 pwd 命令查看当前工作路径)

```
/******创建文件*****/
#include<stdio.h>
#include<fcntl.h>
#define smode 00751
int main(){
    int fd;
    if((fd=creat("/home/filedir/file1",smode))==-1)
        printf("Create file failed!\n");
    else
        printf("Create file OK\n");
    return 0;
}
```

```

/*****读文件*****/
#include<stdio.h>
#include<fcntl.h>
#define rwmode 0
int main(){
    int fd;
    char buffer[1024];
    int n;
    if((fd=open("/home/my.c",rwmode))===-1)
        printf("Can't open file\n");
    else
        printf("Open file OK\n");
    n=read(fd,buffer,sizeof(buffer)-1);
    buffer[n]='\0';
    printf("%s\n",buffer);
    close(fd);
    return 0;
}

```

```

/*****写文件*****/
#include<stdio.h>
#include<fcntl.h>
#define FileSize 1024
#define readmode 0
#define writemode 2
int main(){
    int read_fd,write_fd;
    int readbytes,writebytes;
    char buffer[FileSize],*p;
    if((read_fd=open("/home/my.c",rwmode))===-1){
        printf("Can't open source file\n");
        return -1;
    }
    else{
        printf("Open source file OK\n");
        readbytes=read(read_fd,buffer,FileSize);
        if(readbytes===-1){
            printf("Read file failed\n");
            close(read_fd);
            return -1;
        }
    }
}

```

```
if((write_fd=open("/home/filedir/file1",rwmode))== -1){
    printf("Can't open destination file1\n");
    close(read_fd);
    return -1;
}
else{
    printf("Open destination file OK\n");
    p=buffer;
    writebytes=write(write_fd,p,readbytes);
    if(writebytes== -1){
        printf("Write file failed\n");
        close(read_fd);
        close(write_fd);
        return -1;
    }
}
close(read_fd);
close(write_fd);
return 0;
}
```

参考文献

- [1] 《计算机操作系统》汤子瀛等编著 西安电子科技大学出版社 2001 年
- [2] 《UNIX 操作系统教程—管理与编程》刘循 高等教育出版社 2003 年
- [3] 《计算机操作系统—学习指导与题解》梁红兵等编著 西安电子科技大学出版社 2003 年
- [4] 《操作系统实验指导—基于 Linux 内核》徐虹等编著 清华大学出版社 2004 年
- [5] 《操作系统原理与实例分析》蒲晓蓉等编著 机械工业出版社 2004 年
- [6] 《Linux 操作系统及实验教程》李善平等编著 机械工业出版社 1999 年
- [7] 《操作系统习题与实验指导》左万历 高等教育出版社 2005 年
- [8] 《数据结构、算法与应用—C++语言描述》Sartaj Sahni 著 汪诗林等译 机械工业出版社
2004 年